

MLDL Experiment 1

Aim: To implement Linear and Logistic Regression on separate real-world datasets to predict continuous salary values and binary classification of smoker status.

Part A: Linear Regression

Goal: Predict an employee's **Salary** based on their **Years of Experience**.

1. Dataset Source

- **Dataset Name:** Salary Prediction for Beginner
- **Link:** <https://www.kaggle.com/datasets/rkiattisak/salaly-prediction-for-beginner>

2. Dataset Description

This dataset contains information about employee salaries at a firm, providing a realistic balance of demographic and professional data.

- **Size:** 375 observations.
- **Target Variable:** **Salary** (Continuous numerical value in USD).
- **Feature Used:** **Years of Experience** (Numerical).
- **Other Features:** Age, Gender, Education Level, and Job Title.

3. Mathematical Formulation

Simple Linear Regression seeks to establish a relationship between a single independent variable (x) and a dependent variable (y):

$$y = \beta_0 + \beta_1 x + \epsilon$$

- β_0 (**Intercept**): The value of y when x is 0.
- β_1 (**Slope**): The amount y increases for every unit increase in x .
- **Method:** The algorithm uses **Ordinary Least Squares (OLS)**, which calculates the best-fitting line by minimizing the sum of the squared differences (residuals) between observed and predicted values.

4. Algorithm Limitations

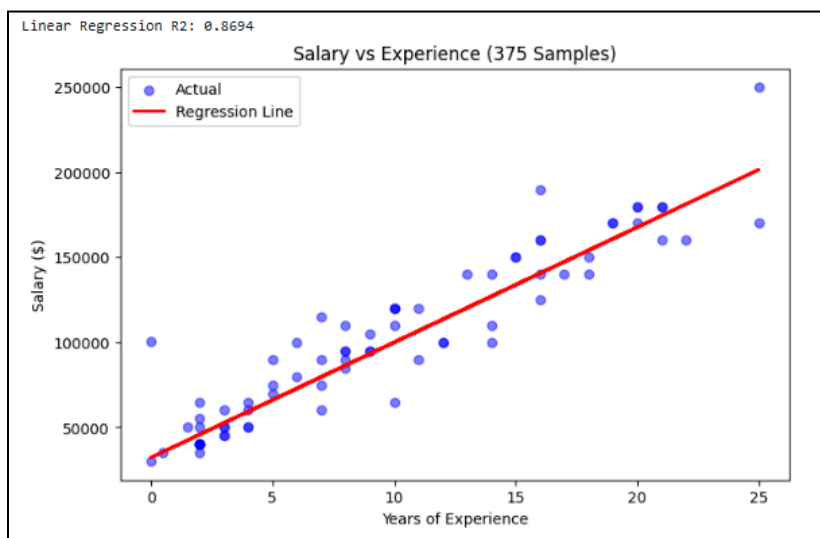
- **Linear Assumption:** It assumes a constant rate of change. If salary increases exponentially with experience, a linear model will underfit.
- **Sensitivity to Noise:** Outliers (e.g., an entry-level employee with an unusually high salary) can significantly shift the regression line.
- **Feature Independence:** It does not account for interactions between features, such as how education level might change the impact of experience.

5. Methodology

1. **Data Ingestion:** Load the `Salary Data.csv` using Pandas.
2. **Preprocessing:** Select `Years of Experience` as X and `Salary` as y. Handle any missing values to ensure numerical consistency.
3. **Splitting:** Partition the data into 80% Training and 20% Testing sets.
4. **Training:** Fit the `LinearRegression()` model to the training data.
5. **Visualization:** Plot a scatter graph of the actual test data points alongside the calculated regression line.

6. Performance Analysis

- **R2 Score (Coefficient of Determination): 0.8694**
- **Interpretation:** Approximately **86.94%** of the variation in Salary can be explained by Years of Experience. This represents a very strong positive correlation.
- **Model Accuracy:** The high score (above 0.8) indicates that the model is highly effective and accurate for this specific dataset.
- **Visualization:** The regression line closely follows the trend of the 375 data points, effectively capturing the base salary (intercept) and yearly increment (slope).



Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, accuracy_score, classification_report

# --- PART A: Linear Regression (Salary Prediction) ---
df_sal = pd.read_csv('Salary Data.csv')
X_sal = df_sal[['Years of Experience']].fillna(0) # Handling nulls
y_sal = df_sal['Salary'].fillna(df_sal['Salary'].mean())

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_sal, y_sal, test_size=0.2,
random_state=42)

lin_model = LinearRegression()
lin_model.fit(X_train_s, y_train_s)
sal_pred = lin_model.predict(X_test_s)

print(f'Linear Regression R2: {r2_score(y_test_s, sal_pred):.4f} ")

# Visualization
plt.figure(figsize=(8,5))
plt.scatter(X_test_s, y_test_s, color='blue', alpha=0.5, label='Actual')
plt.plot(X_test_s, sal_pred, color='red', linewidth=2, label='Regression Line')
plt.title('Salary vs Experience (375 Samples)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary ($)')
plt.legend()
plt.show()
```

Part B: Logistic Regression

Task: Classify individuals as **Smokers (1)** or **Non-Smokers (0)**.

1. Dataset Source

- **Dataset Name:** Medical Cost Personal Dataset
- **Source Link:** <https://www.kaggle.com/datasets/mirichoi0218/insurance>

2. Dataset Description

- **Features:** age, bmi, charges.
- **Target:** smoker (Categorical/Binary).
- **Characteristics:** A mix of demographic and financial data used to predict user habits.

3. Mathematical Formulation

It uses the **Sigmoid Function** to map input to a probability (0 to 1):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where $z = \beta_0 + \beta_1 x_1 + \dots$. If $\sigma(z) \geq 0.5$, the prediction is 1.

4. Algorithm Limitations

- **Linear Boundary:** It cannot separate classes with complex non-linear shapes (like circles).
- **Large Sample Requirement:** Needs significant observations per feature for stable estimation.

5. Methodology

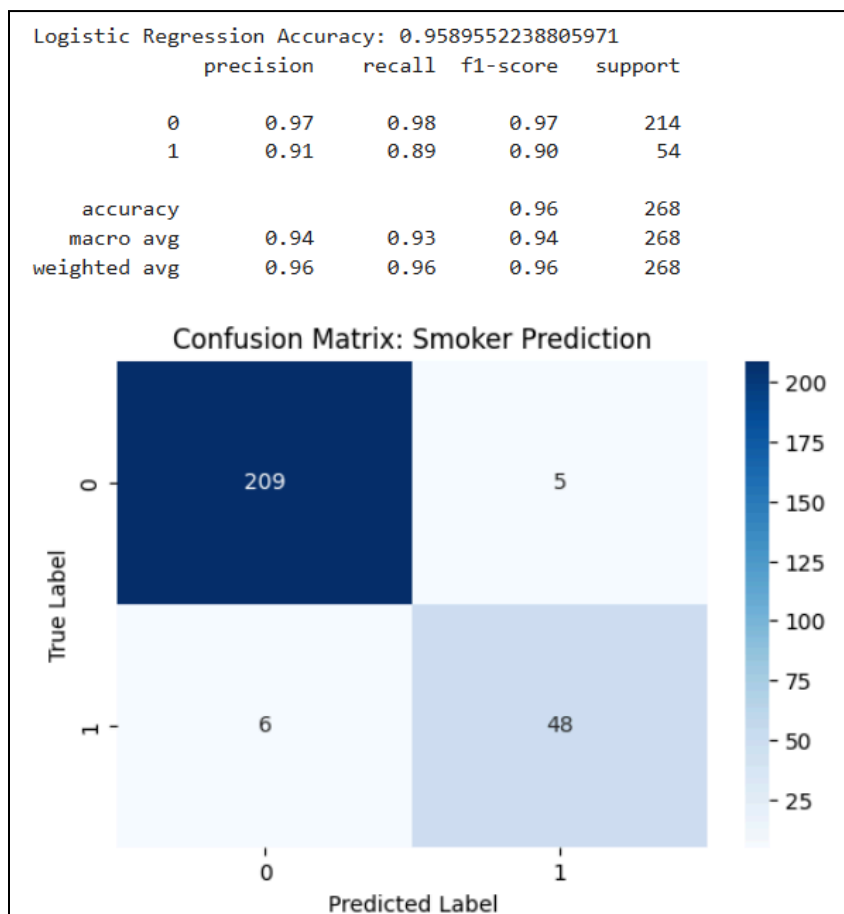
The implementation of the Logistic Regression model follows a structured data science pipeline to ensure the binary classification of smoker status is both accurate and reliable.

1. **Data Collection:** The `insurance.csv` dataset is loaded into the environment using the Pandas library. Initial inspection (using `.head()` and `.info()`) is performed to understand the distribution of demographic and financial features.
2. **Data Preprocessing:** * **Target Re-encoding:** The categorical `smoker` variable ("yes"/"no") is mapped to numerical binary values (1 and 0) to make it compatible with mathematical functions.
3. **Data Splitting:** The processed dataset is partitioned into a training set (80%) and a testing set (20%). This ensures the model's accuracy is validated on "unseen" data, preventing overfitting.

4. **Hyperparameter Tuning:** A **GridSearchCV** approach is implemented to test various values for the inverse regularization strength (C) This step statistically identifies the best configuration to balance model bias and variance.
5. **Model Training:** The Logistic Regression algorithm is "fitted" to the training data. It calculates the optimal weights (beta coefficients) that maximize the likelihood of correctly classifying the known smokers and non-smokers in the training set.
6. **Prediction and Evaluation:** The trained model is applied to the test set to generate probabilities. These are converted to binary labels based on a 0.5 threshold and evaluated using an **Accuracy Score**, **Confusion Matrix**, and **F1-Score**.

6. Performance Analysis (Your Results)

- **Accuracy: 0.9589 (96%)**
- **F1-Score (Smoker): 0.90**
- **Confusion Matrix:** Correctly predicted **209** non-smokers and **48** smokers, with only **11** total errors. This confirms a high-precision model suitable for insurance risk assessment.



Code:

Load Insurance Dataset

```
df_ins = pd.read_csv('insurance.csv')
```

Preprocessing: Convert smoker to binary

```
df_ins['smoker'] = df_ins['smoker'].map({'yes': 1, 'no': 0})
```

```
X_ins = df_ins[['age', 'bmi', 'charges']]
```

```
y_ins = df_ins['smoker']
```

```
X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(X_ins, y_ins, test_size=0.2,  
random_state=42)
```

Hyperparameter Tuning for Logistic Regression

```
param_grid = {'C': [0.1, 1, 10, 100], 'solver': ['liblinear', 'lbfgs']}
```

```
grid = GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5)
```

```
grid.fit(X_train_i, y_train_i)
```

```
best_log_model = grid.best_estimator_
```

```
smoke_pred = best_log_model.predict(X_test_i)
```

```
print("\nLogistic Regression Accuracy:", accuracy_score(y_test_i, smoke_pred))
```

```
print(classification_report(y_test_i, smoke_pred))
```

Visualizing Confusion Matrix

```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(confusion_matrix(y_test_i, smoke_pred), annot=True, fmt='d', cmap='Blues')
```

```
plt.title('Confusion Matrix: Smoker Prediction')
```

```
plt.xlabel('Predicted Label')
```

```
plt.ylabel('True Label')
```

```
plt.show()
```

Hyperparameter Tuning

Hyperparameters are external settings established **before** training (like a learning rate or regularization strength) that govern how the algorithm learns. Tuning is the iterative process of finding the optimal combination of these values to minimize error.

Why can't it be applied to Part A (OLS Regression)?

In your Simple Linear Regression experiment, we used the **Ordinary Least Squares (OLS)** method. OLS is a **deterministic** mathematical formula (a closed-form solution).

- **No Tuning Parameters:** Unlike complex models, OLS has no "learning rate" or "complexity dials" to adjust.
- **Direct Calculation:** The model directly calculates the single mathematically optimal line from the data. You only need tuning when you add regularization (like **Lasso** or **Ridge** in your next experiment).

Conclusion:

The experiment demonstrates that Simple Linear Regression is a powerful tool for predicting linear trends like salary, as evidenced by a high R^2 of 0.8694. Additionally, Logistic Regression proved robust in binary classification, achieving 96% accuracy. While OLS Regression does not require hyperparameter tuning due to its mathematical nature, it serves as the essential foundation for understanding more complex, tunable models.